

Design Document — Binary-Outcome Order Matching Engine

This document explains how the engine is built and, more importantly, *why* each decision went the way it did — including the two cases where I shipped the wrong design to myself first, reproduced the failure, and corrected it. Where a choice has a real cost, I name the cost. The decisions are mine; Claude typed and pressure-tested them.

1. Problem and scope

The task is a matching engine for one binary-outcome market with a REST API. A binary outcome (YES/NO) means a price *is* a probability, so prices are integer cents in `[1, 99]` — a small, fixed domain, which influences several data-structure choices below. The engine matches limit orders, rests unfilled remainder, supports cancellation, and exposes the book.

I scoped to a single book and the named edge cases plus two extras. I deliberately did **not** build IOC/FOK order types or multi-instrument support. Section 11 explains why stopping was a decision, not an omission.

2. Domain model — what an order *is*

An `Order` has six fields fixed at creation and exactly one that changes:

- **Client-supplied, frozen:** `side`, `price`, `quantity`, `owner_id`.
- **Engine-assigned, frozen:** `order_id`, `sequence_number`.
- **Mutable:** `remaining`.

Why only `remaining` is mutable, and why it's mutable at all. The book stores resting orders *by reference* — a price level holds the actual `Order` object, and cancellation/removal finds it by identity. When an order partially fills, I mutate `remaining` in place so the reference the book holds is still the live order. The obvious-looking alternative — make the whole model frozen and construct a fresh `Order` on every fill — would orphan every structure that points at the old object, forcing me to re-thread the deque and the index on each partial fill. In-place mutation of one field is the cheaper, less error-prone design. The cost is that `Order` can't be a plain frozen model.

Why Pydantic v2 frozen wasn't enough. Pydantic v2's `frozen=True` is all-or-nothing at the model level; it can't freeze five fields and leave one writable. So I enforce the boundary myself with a `__setattr__` override and a `_FROZEN` set: writing a frozen field raises `AttributeError`, and writes to `remaining` are guarded to stay an integer in `[0, quantity]`. This is a runtime guard, not a type-checker guarantee — mypy won't catch `order.price = 50` — and the docstring says so.

sequence_number is a counter, not a wall clock. Time priority needs a strict total order over arrivals. A wall-clock timestamp gives neither: two orders can land in the same microsecond (ambiguous ties), and clocks can move backward (NTP correction), which would corrupt FIFO. A monotonic integer counter incremented once per accepted order is unconditionally strictly increasing and ties-free. `order_id` is derived as `f"order-{sequence_number}"`. The cost is that the ordering is per-process, not globally meaningful across machines — which is the right tradeoff for a single-writer engine and is exactly what I'd revisit when sharding (section 6).

Prices are integer cents 1–99. A binary contract's price is a probability; representing it as an integer cent avoids floating-point comparison hazards in the matching loop entirely, and the `[1, 99]` bound is enforced by a Pydantic `Field` constraint, so invalid prices are rejected at the edge before they ever reach the engine.

3. The order book — data structure and its runtime

bids: dict[int, deque[Order]] # price -> FIFO queue of resting buys

asks: dict[int, deque[Order]] # price -> FIFO queue of resting sells

index: dict[str, Order] # order_id -> Order, for O(1) cancel lookup

Each price level is a `deque`, which gives FIFO within a level for free: append on the right, match from the left. The flat `index` lets cancellation find any order by id without scanning.

Runtime, stated honestly:

Operation	Cost	Why
<code>add</code>	$O(1)$ amortized	dict insert + deque append
<code>cancel</code>	$O(1)$ lookup + $O(k)$ removal	<code>deque.remove</code> is linear in level depth <code>k</code>

Operation	Cost	Why
<code>prices</code>	$O(P \log P)$	sorts the price keys on demand (P = number of distinct levels)

Why a plain `dict` sorted on demand, not a `SortedDict`. This is where the `[1, 99]` price domain matters. There are at most 99 levels per side, so sorting the keys is $O(99 \log 99)$ — effectively constant. A `SortedDict` would make best-price access $O(1)$ and avoid the re-sort, but for a 99-level domain it buys almost nothing and adds a dependency. If this were an instrument with a large or unbounded price domain, the re-sort would dominate and I'd switch to a sorted container immediately. The choice is correct *for this instrument*, and I'd flag it the moment the domain changed.

The $O(k)$ cancel is the one I'd fix first. `deque.remove` walks the level to find the order. Replacing each level with an **intrusive doubly-linked list** — where the `Order` carries its own prev/next pointers — makes removal $O(1)$. I didn't do it here because it touches five surfaces (add, remove, level iteration, snapshot, the index) and introduces a real failure mode of its own: dangling index entries if a node is unlinked but not removed from the index. That's a refactor to do deliberately with tests, not to rush into a take-home. It's the top item in future work.

4. The matching algorithm

`_match` walks the opposite side best-price-first and fills until the incoming order is exhausted or the price stops being acceptable:

for price in best-first order on the opposite side:

 stop if incoming is fully filled

 stop if price is worse than the incoming order's limit

 (buy: price > limit; sell: price < limit)

 for each resting order at this price (FIFO):

 ... fill min(incoming.remaining, resting.remaining) ...

Three details worth calling out:

- I **iterate a copy of each level** (`list(level)`) because matching removes filled orders from the same deque mid-loop; iterating the live deque while mutating it is a bug waiting to happen.
- **Trade direction is derived from the aggressor.** The incoming order is the aggressor; its side determines which of the two ids is the buy id and which is the sell id on the `Trade`. The fill always prints at the *resting* order's price (the aggressor crosses to meet the book), which is what `test_better_price_beats_earlier_time` pins down: a resting sell at 50 fills a buy limit of 52 at **50**, not 52.
- **Unfilled remainder rests.** After the loop, if `remaining > 0`, the order is added to the book at its own price.

After every submit I assert the **no-crossed-book invariant**: the best bid must never be $\geq$ the best ask. A crossed book (a bid priced above an ask) is logically impossible in a correct engine — those orders should have matched. Violating it raises `BookInvariantError`. Importantly this is a **raise, not an assert**: `assert` statements are stripped under `python -O`, which would silently disable the safety net in exactly the production builds where you most want it.

5. Self-trade prevention — the first bug I shipped to myself

A single participant can legitimately rest a buy and a sell at the same time, as long as they don't cross in price. The problem is only when a *new* order would match against that *same owner's* resting order.

My first implementation used skip-and-continue: on a self-match, skip that resting order and keep scanning. It looked clean. It was wrong, and I only caught it because I probed the book state instead of trusting that the code read correctly. After a same-owner submit sequence, the book showed `best_bid = 60, best_ask = 55` — a **crossed book**.

The reason is subtle and worth stating precisely: skip-and-continue leaves the owner's resting order *in place* while the aggressor *also* comes to rest. Now both sit on the book on opposite sides at crossing prices — best bid above best ask — because neither was allowed to consume the other. The skip didn't prevent the cross; it manufactured it.

The fix is cancel-resting: on a self-match, *remove* the owner's resting order from the book, then continue. The order that would have crossed is gone, so the aggressor's remainder can rest without producing a cross. This is safe by construction, not by luck — there is no longer an order left behind to cross against. The test suite pins this down from several angles: self-match at the head of a FIFO queue, in the middle of a queue between two other participants, and with multiple of the owner's own orders at one level

```
(test_self_trade_skip_in_middle_of_fifo,
test_self_trade_multiple_own_orders_same_level).
```

The cost of cancel-resting is that an aggressive self-crossing order wipes out the participant's own resting liquidity in its path. For a self-trade-prevention policy that's the correct and conventional behavior; the alternative policies (cancel-incoming, decrement-both) trade one surprise for another, and I chose the one that cannot leave the book in an invalid state.

6. Concurrency — the second bug I shipped to myself

This is the decision I most want to get right on the record, because my *first* answer here was wrong in an instructive way.

The wrong first answer. I reasoned that if the engine is single-threaded, the cancel-while-matching race can't exist by construction, so no lock is needed and a concurrency test would be contradictory. I removed the lock. That reasoning has a hole: **the engine isn't single-threaded just because I want it to be**. FastAPI runs synchronous `def` path operations in an anyio threadpool, so concurrent HTTP requests execute engine code on *different threads* — the framework hands me concurrency whether I asked for it or not.

The race, concretely. Thread A is mid-match, iterating a level's deque. Thread B's `cancel_order` calls `remove`, which mutates that same deque **and** the index in two separate steps. Without serialization, Thread A can observe a half-removed order — present in the deque but gone from the index, or vice versa — and the operation throws. I reproduced this under concurrent load with actual probes; it is not theoretical.

Why the GIL doesn't save me. Python's GIL makes individual bytecodes atomic, so a single `dict` write or `deque` append can't tear. What it does *not* protect is a multi-step composite operation like "remove from deque, then pop from index" — the GIL can be released between those two statements, letting another thread in mid-mutation. The race lives in the gap *between* bytecodes, which is exactly what a lock closes.

The fix. A single `threading.RLock` wraps `submit_order`, `cancel_order`, `resolve_market`, and `snapshot`. Now at most one operation is inside the engine at a time — it behaves as a **single writer** regardless of how many threads the framework throws at it. I used `RLock` rather than `Lock` as cheap insurance: a re-entrant lock won't self-deadlock if I later add a feature like cancel-replace where one locked method needs to call another. It survives any deployment and it's three lines.

The honest tradeoff — single-writer vs. multithreaded. The decision only means something if I can name what I gave up.

Single-writer *buys* me:

- **Determinism** — the same sequence of orders always produces the same trades in the same order, which is what makes the system auditable and replayable.
- **Correctness by construction** — the cancel-while-matching race can't *exist*; I'm eliminating it, not managing it.
- **Exact FIFO fairness** — there is one well-defined order of operations.

Single-writer costs me:

- **No multi-core parallelism inside a book.** Throughput is bounded by one core, and under heavy load requests queue behind the lock. I cannot make one book faster by adding threads.

Multithreading would buy parallel matching and higher throughput on many cores, at the cost of non-determinism (trade order would depend on the scheduler — unacceptable where fairness and replayability matter) and a class of intermittent, load-only bugs that are miserable to reproduce — precisely the race I already hit, but worse.

The verdict, and the scaling answer. For a matching engine, determinism and correctness dominate raw throughput — this is how real exchanges run their core, single-threaded, for the same reason. The right way to scale is *not* to parallelize within a book; it's to keep each book single-writer and deterministic and **shard by instrument**, so every contract runs on its own engine, with a central sequencer only if global cross-shard ordering is ever needed. That is the answer I'd give to "how does this scale?" — and it's the opposite of the instinct to add threads.

What the concurrency test actually proves. `test_concurrency_stress` runs 8 threads × 250 submits against overlapping prices. It *cannot* guarantee it catches a race — race windows are narrow and non-deterministic. What it *can* prove is that under heavy concurrent access no exception is raised and the structural invariants hold after all threads join (every id in the index sits in exactly one level, and every order in a level is in the index). Run lock-less, that test reliably surfaces the two-step-removal corruption as a thrown exception. What it does **not** catch is subtle reordering that produces wrong trades without throwing — that would need a deterministic scheduler. The test catches corruption, not logical incorrectness under concurrency, and I'd describe it that way rather than overclaim.

7. Idempotency — and why the key is generated client-side

`submit_order` is made idempotent with a `client_order_id`: the first submission with a given id is processed and its `SubmitResult` cached; any retry with the same id returns the identical result without creating a second order.

Why the client mints the id, not the server. Idempotency exists to protect a client that *doesn't know whether its request succeeded* — the response was lost to a timeout, say. To retry safely the client must send the *same* key, and that's only possible if the client generated it *before* the network call. If the server minted the id, a retry would arrive with no id (or a new one), the server couldn't recognize it as a duplicate, and the retry would place a second order — defeating the entire purpose. So client-side generation isn't a style choice; it's a requirement for the guarantee to hold.

The limitation, stated up front. The cache is unbounded — it grows for the life of the process. That's fine for an exercise and a leak in production; the fix is TTL- or size-based eviction (an LRU), which fits the single-writer model cleanly since the writer is the natural place to evict. A second, smaller gap: the cache trusts the key and doesn't verify that a retried request *body* matches the original, so reusing a key with different fields silently returns the first result. A stricter version would hash the request and reject mismatches.

8. Market resolution

When a binary outcome resolves, `resolve_market(outcome)` voids every open order, records the outcome, and blocks further submissions — any later submit raises `MarketResolvedError`.

Why void rather than settle. Once the outcome is known, open limit orders are meaningless and must not execute; voiding them is the minimal correct behavior. *Settlement* — paying out position holders at the resolved price — is a separate clearing concern and out of scope for a matching engine.

Why it's engine-level, not an HTTP endpoint. Resolution is a privileged operational action (an oracle or admin resolves the market), not something a participant does. Exposing it as an unauthenticated `POST` would be wrong, and there's no auth layer in scope, so I modelled and tested resolution at the engine and left the public endpoint for the version that has authentication. That endpoint is in future work, behind auth.

9. Error handling and status codes

- `201` on submit — a resource was created.
- `404` on a cancel miss. The engine returns `None` both when the order doesn't exist *and* when it exists but belongs to another owner — deliberately not distinguishing the two, so a non-owner can't probe for the existence of someone else's order.

- **422** from Pydantic on invalid input (bad price, non-positive quantity, bad owner), produced at the edge before the engine runs.
- **BookInvariantError** is a **raise**, not an **assert** — see section 4; **assert** vanishes under **-O**.

Known gap, named rather than hidden: **app.py** does not yet map **MarketResolvedError** or **BookInvariantError** to HTTP status codes, so a post-resolution submit currently surfaces as a **500** rather than a **409 Conflict**. The transport layer is intentionally thin; adding exception handlers is the right fix and is in future work.

10. Testing strategy

37 tests, all in-process against the engine and book with no web server, which is the payoff of keeping **app.py** as the only HTTP-aware module. Coverage is organized around the things that can actually break:

- **Matching:** full fill, partial fill (incoming rests / resting stays), multi-level sweep, sell-side aggressor, fill-accounting invariants.
 - **Priority:** asks ascending, bids descending, better-price-beats-earlier-time, FIFO within a level, FIFO preserved across a cancellation, partially-filled resting order keeps its place.
 - **Self-trade:** at the head of a queue, in the middle between other participants, multiple own orders at a level, and three "no crossed book" checks.
 - **Cancellation:** removes a resting order, wrong owner rejected, nonexistent returns **None**, already-filled returns **None**, returns remaining-at-cancel-time.
 - **Validation / immutability:** price and quantity bounds, owner validation, duplicate id rejected, returned **Trade** and **CancelledOrder** snapshots can't be mutated.
 - **Extras:** idempotent submit returns the cached object, market resolution voids all and blocks new submits, and the 8-thread concurrency stress test.
-

11. Scope discipline — what I left out, on purpose

I stopped at the four named edge cases plus two extras (idempotency, resolution). I did **not** add IOC (immediate-or-cancel) or FOK (fill-or-kill) order types. They multiply the order state machine — FOK requires pre-checking total fillability before executing anything, IOC requires cancel-the-remainder semantics — for marginal signal on a take-home. Adding them would demonstrate the *opposite* of what I want to show: I'd rather submit four well-understood, well-tested behaviors than ten half-reasoned ones. Knowing where to stop is part of the work.

12. Future work, consolidated

In rough priority order:

1. **Map engine exceptions to HTTP codes** (409 for resolution-closed; 500-with-alert for invariant violations).
 2. **Bound the idempotency cache** with TTL or LRU eviction.
 3. **Intrusive doubly-linked list per price level** for true $O(1)$ cancellation.
 4. **Authentication**, replacing the `owner_id` stand-in, and an authenticated **resolution endpoint**.
 5. **GET /orders/{order_id}** for live single-order status.
 6. **Dual YES/NO book** with cross-side matching — the binary-market-correct structure.
 7. **Sharding per instrument** as the scaling path — single-writer engines partitioned by contract, with a central sequencer only if global ordering is needed. Not concurrency inside a book.
-

13. How I worked

I used Claude as an implementation assistant — to type and pressure-test code against decisions I'd already made — not as a designer. Every choice here is mine: the data structure and its runtime tradeoff, the counter-not-clock timing decision, the single-writer concurrency model and the two options I weighed to enforce it, the integer-cent prices, the status-code mappings, and the scoping to a single book. The self-trade and concurrency sections in particular reflect real debugging and a corrected assumption — I shipped skip-and-continue self-trade prevention and a lock-less engine to myself first, reproduced the crossed book and the threadpool race with actual probes, and fixed both. That is exactly the kind of reasoning a one-shot prompt doesn't produce, and the accompanying AI-usage export shows the process directly.